

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

ЗВІТ

до лабораторної роботи № 5

з дисципліни «Програмування мікроконтролерів»

Студента 2-го курсу
спеціальності 123 –
«Комп'ютерна інженерія»
Євтушика Івана Івановича

м. Ужгород – 2026 р.

Лабораторна робота №5

Тема: Динамічна індикація, таймери, робота з логічним аналізатором у інтегрованому середовищі *PSoC® Creator™*.

Мета: Ознайомитися з принципами роботи динамічної індикації та налаштуванні переривань у мікроконтролерних системах на базі *PSoC*. Набути практичних навичок створення проєктів у інтегрованому середовищі *PSoC® Creator™*, налаштування апаратних компонентів для керування індикаторами через зсувні регістри та реалізації ефективного виведення даних. Також набути практичних навичок використання логічного аналізатора на прикладі *Saleae Logic*.

Хід виконання завдань

Варіант 3

Завдання 1. Вивести на дисплей число та декодувати його за допомогою логічного аналізатора.

Відповідно до умови лабораторної роботи, число для виведення на семисегментний індикатор визначається за формулою:

Число = номер по списку в загальній групі + теперішній рік у форматі двох значень

Для мого варіанту:

Номер по списку = 3 Рік = 26

Отже: $3 + 26 = 29$

Тому на семисегментний світлодіодний індикатор потрібно вивести число 29 та перевірити передавання даних за допомогою логічного аналізатора.

Код:

```
#include "project.h"

#define DIGIT_2_CODE 0xA4
#define DIGIT_9_CODE 0x90

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1);
    }
}
```

```

        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneCode(uint8_t position_code,
uint8_t segment_code)
{
    FourDigit74HC595_sendData(position_code);
    FourDigit74HC595_sendData(segment_code);

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

int main(void)
{
    CyGlobalIntEnable;

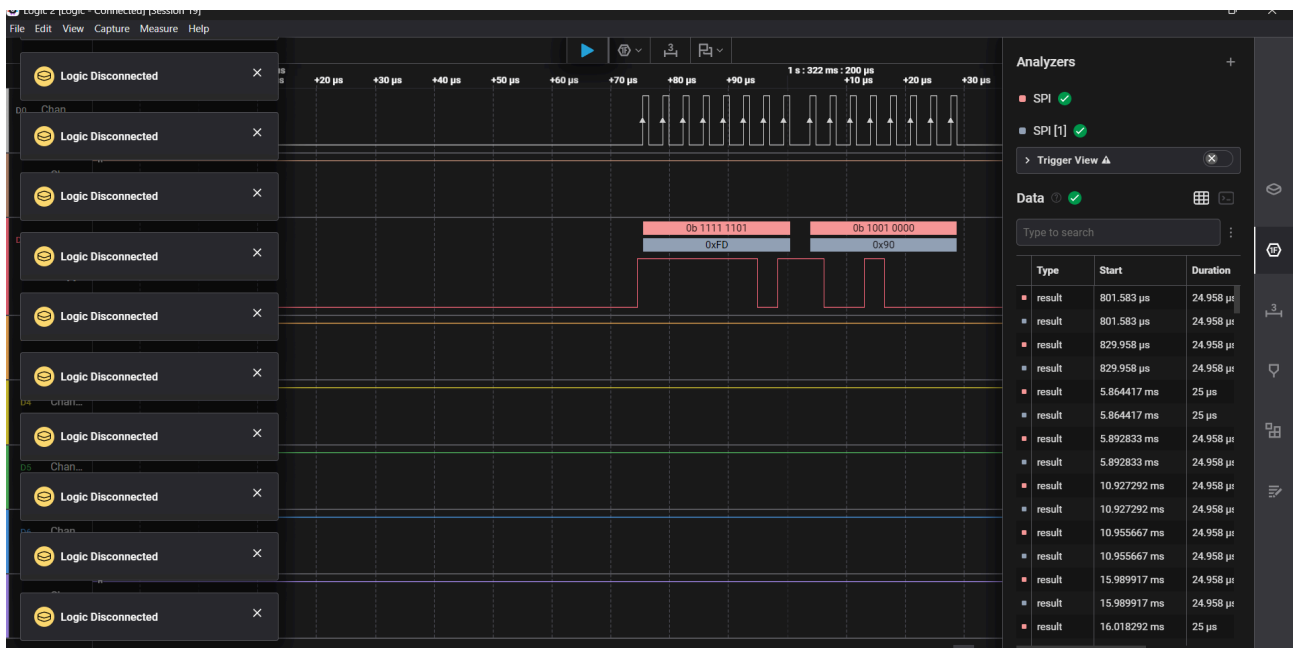
    for (;;)
    {
        FourDigit74HC595_sendOneCode(0xFE, DIGIT_2_CODE);
        CyDelay(5);

        FourDigit74HC595_sendOneCode(0xFD, DIGIT_9_CODE);
        CyDelay(5);
    }
}

```

У програмі реалізовано виведення числа 29 на семисегментний індикатор. Для цього використовуються коди цифр 2 та 9, які передаються у зсувний регістр. Завдяки динамічній індикації цифри по чергово виводяться на відповідні позиції індикатора, але для користувача вони виглядають як одне стабільне число.

Логічний аналізатор:



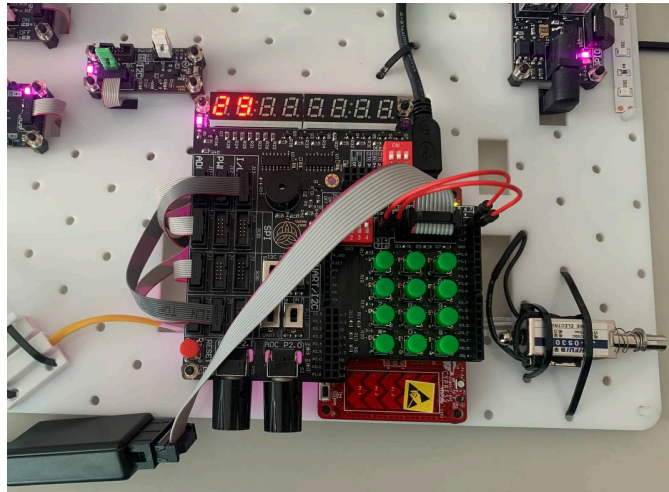
Для перевірки правильності передавання даних було використано логічний аналізатор *Saleae Logic*. У програмі *Logic* було налаштовано декодування сигналів у режимі *SPI*. Для аналізу було вибрано відповідні канали сигналів *MOSI* та *Clock*.

Декодування виконувалося у двох форматах:

Hexadecimal -- для перегляду переданих байтів у шістнадцятковому форматі;

Binary -- для перегляду цих самих байтів у двійковому форматі.

Результат: У результаті виконання завдання на семисегментному світлодіодному індикаторі було виведено число 29. За допомогою логічного аналізатора було виконано декодування переданих даних у форматах *HEX* та *Binary*. Отримані значення підтверджують, що дані до зсувного регістра передаються коректно, а цифри 2 і 9 правильно відображаються на дисплеї.



Завдання 2. Додати можливість вводу паролю на семисегментний дисплей, після натиску на кнопку «#». Перевірити введений пароль з тим, який Ви оголосите самостійно в коді, якщо пароль вірний, виконати якусь дію на семисегментних індикаторах (може бути мигання або на ваш розсуд інша індикація). Також якщо пароль не вірний, потрібно сповістити користувача іншим паттерном мигання на семисегментних індикаторах.

Код:

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80,
0x90,
    0xBF, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E, 0x7F,
    0xFF
};

static uint8_t password[4] = {1, 2, 3, 4};
static uint8_t input[4] = {18, 18, 18, 18};

static uint8_t display_data[8] = {
    18, 18, 18, 18, 18, 18, 18, 18
};

static uint8_t key_map[4][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {16, 0, 17}
};

void (*write_col[3])(uint8) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
```

```

        COLUMN_2_Write
    };

void (*set_col_dm[3])(uint8) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};

uint8 (*read_row[4])(void) = {
    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

static volatile uint8 led_counter = 0;
static uint8 input_counter = 0;

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position,
uint8_t digit, uint8_t dot)
{
    if (position >= 8)
    {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        Pin_Latch_Write(1);
        Pin_Latch_Write(0);
        return;
    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (dot)
    {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    }
    else
    {
        FourDigit74HC595_sendData(LED_NUM[digit]);
    }

    Pin_Latch_Write(1);
}

```

```

        Pin_Latch_Write(0);
    }

    CY_ISR(Timer_Int_Handler2)
    {
        Timer_1_ReadStatusRegister();

        FourDigit74HC595_sendOneDigit(led_counter,
display_data[led_counter], 0);

        led_counter++;

        if (led_counter > 7)
        {
            led_counter = 0;
        }
    }

static uint8_t getKey(void)
{
    for (uint8_t c = 0; c < 3; c++)
    {
        for (uint8_t i = 0; i < 3; i++)
        {
            set_col_dm[i](CY_SYS_PINS_DM_DIG_HIZ);
        }

        set_col_dm[c](CY_SYS_PINS_DM_STRONG);
        write_col[c](0);

        for (uint8_t r = 0; r < 4; r++)
        {
            if (read_row[r]() == 0)
            {
                CyDelay(20);

                while (read_row[r]() == 0)
                {
                }

                return key_map[r][c];
            }
        }
    }

    return 255;
}

static uint8_t checkPassword(void)
{
    for (uint8_t i = 0; i < 4; i++)
    {
        if (input[i] != password[i])

```

```

        {
            return 0;
        }
    }

    return 1;
}

static void clearDisplay(void)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        display_data[i] = 18;
    }
}

static void correctPassword(void)
{
    for (uint8_t j = 0; j < 3; j++)
    {
        for (uint8_t i = 0; i < 8; i++)
        {
            display_data[i] = 8;
        }

        CyDelay(250);

        clearDisplay();

        CyDelay(250);
    }
}

static void wrongPassword(void)
{
    for (uint8_t j = 0; j < 3; j++)
    {
        for (uint8_t i = 0; i < 8; i++)
        {
            display_data[i] = 0;
        }

        CyDelay(150);

        clearDisplay();

        CyDelay(150);
    }
}

int main(void)
{
    CyGlobalIntEnable;

```



```

Timer_Int_StartEx(Timer_Int_Handler2);
Timer_1_Start();

for (;;)
{
    uint8_t key = getKey();

    if (key != 255)
    {
        if (key <= 9)
        {
            if (input_counter < 4)
            {
                input[input_counter] = key;
                display_data[input_counter] = key;
                input_counter++;
            }
        }

        if (key == 17)
        {
            if (input_counter == 4)
            {
                if (checkPassword())
                {
                    correctPassword();
                }
                else
                {
                    wrongPassword();
                }
            }

            input_counter = 0;

            for (uint8_t i = 0; i < 4; i++)
            {
                input[i] = 18;
            }

            clearDisplay();
        }

        if (key == 16)
        {
            input_counter = 0;

            for (uint8_t i = 0; i < 4; i++)
            {
                input[i] = 18;
            }
        }
    }
}

```

```

        clearDisplay();
    }
}
}

```

У програмі задано пароль: `static uint8_t password[4] = {1, 2, 3, 4};`

Масив `display_data[8]` відповідає за те, які символи відображаються на семисегментному індикаторі. Значення 18 використовується для очищення позиції індикатора. Зчитування клавіш виконується за допомогою функції `getKey()`. У ній по черзі активуються стовпці клавіатури та перевіряється стан рядків. Якщо клавіша натиснута, функція повертає її значення.

Після введення чотирьох цифр користувач натискає кнопку «#». У програмі вона має значення 17. Після цього викликається функція `checkPassword()`, яка порівнює введений пароль із заданим у коді.

Якщо пароль правильний, виконується функція `correctPassword()`. У ній усі індикатори кілька разів заповнюються цифрою 8, після чого очищаються. Якщо пароль неправильний, виконується функція `wrongPassword()`, де індикатори блимають іншим способом – через виведення цифри 0.

Кнопка «*» має значення 16 і використовується для очищення введення та скидання дисплея.

Результат: У результаті виконання завдання було реалізовано введення паролю з матричної клавіатури на семисегментний індикатор. Після введення чотирьох цифр і натискання кнопки «#» програма перевіряє пароль.

Якщо введено правильний пароль 1234, семисегментні індикатори виконують індикацію правильного введення – кілька разів блимають цифрою 8. Якщо пароль введено неправильно, виконується інший патерн мигання -- індикатори блимають цифрою 0. Це дозволяє користувачу зрозуміти, чи був пароль введений правильно.

Завдання 3. Виконати циклічний вивід певного набору символів на семисегментних індикаторах за варіантами:

3	Перші чотири вправо, наступні фіксовані
---	---

Код:

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80,
0x90,
    0xBF, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E, 0x7F,
    0xFF
};

static uint8_t display_data[8] = {
    1, 2, 3, 4, 5, 6, 7, 8
};

static volatile uint8 led_counter = 0;
static volatile uint16 shift_counter = 0;

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position,
uint8_t digit, uint8_t dot)
{
    if (position >= 8)
    {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        Pin_Latch_Write(1);
        Pin_Latch_Write(0);
        return;
    }
}
```

```

    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (dot)
    {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    }
    else
    {
        FourDigit74HC595_sendData(LED_NUM[digit]);
    }

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

static void shiftFirstFourRight(void)
{
    uint8_t temp = display_data[3];

    display_data[3] = display_data[2];
    display_data[2] = display_data[1];
    display_data[1] = display_data[0];
    display_data[0] = temp;
}

CY_ISR(Timer_Int_Handler2)
{
    Timer_1_ReadStatusRegister();

    FourDigit74HC595_sendOneDigit(led_counter,
display_data[led_counter], 0);

    led_counter++;

    if (led_counter > 7)
    {
        led_counter = 0;
    }

    shift_counter++;

    if (shift_counter >= 500)
    {
        shift_counter = 0;
        shiftFirstFourRight();
    }
}

int main(void)
{
    CyGlobalIntEnable;

```

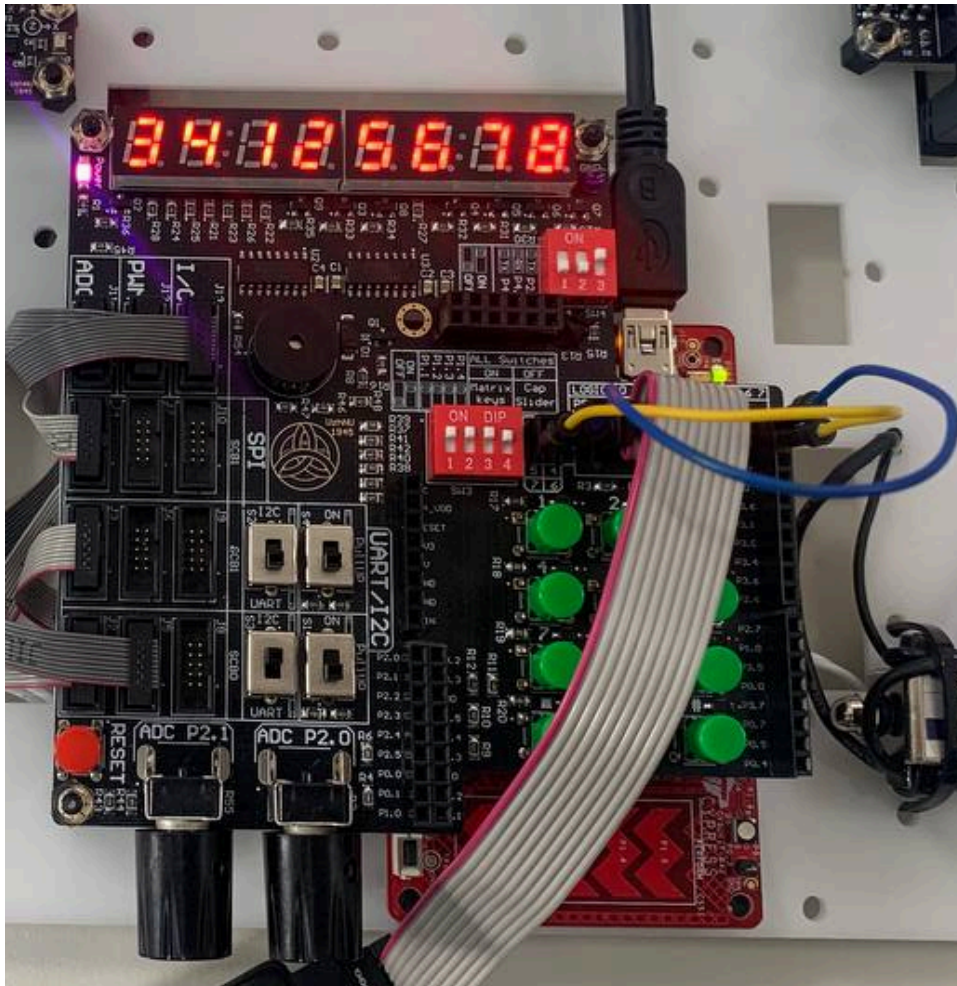
```

Timer_Int_StartEx(Timer_Int_Handler2);
Timer_1_Start();

for (;;)
{
}
}

```

Результат: У результаті виконання програми на семисегментному індикаторі відображається послідовність цифр 1 2 3 4 5 6 7 8. Перші чотири цифри циклічно зсуваються вправо, а останні чотири цифри 5 6 7 8 залишаються фіксованими.



Функція *shiftFirstFourRight()* виконує зсув тільки перших чотирьох елементів масиву *display_data*. Змінна *shift_counter* задає проміжок часу між зсувами. Завдяки цьому на індикаторі видно, що перші чотири символи рухаються вправо, а наступні залишаються без змін.

Завдання 4. На перших чотирьох індикаторах реалізувати відлік 5

ХВИЛИН.

Код:

```
#include "project.h"

static uint8_t LED_NUM[] = {
    0xC0, 0xF9, 0xA4, 0xB0, 0x99, 0x92, 0x82, 0xF8, 0x80, 0x90,
    0xBF, 0x88, 0x83, 0xC6, 0xA1, 0x86, 0x8E, 0x7F,
    0xFF
};

static uint8_t display_data[8] = {
    0, 5, 0, 0, 18, 18, 18, 18
};

static volatile uint8 led_counter = 0;
static volatile uint16 ms_counter = 0;

static uint16 seconds = 300;

static void FourDigit74HC595_sendData(uint8_t data)
{
    for (uint8_t i = 0; i < 8; i++)
    {
        Pin_DO_Write((data & (0x80 >> i)) ? 1 : 0);
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position,
uint8_t digit, uint8_t dot)
{
    if (position >= 8)
    {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
        Pin_Latch_Write(1);
        Pin_Latch_Write(0);
        return;
    }

    FourDigit74HC595_sendData(0xFF & ~(1 << position));

    if (dot)
    {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    }
    else
    {
        FourDigit74HC595_sendData(LED_NUM[digit]);
    }
}
```

```

    }

    Pin_Latch_Write(1);
    Pin_Latch_Write(0);
}

static void updateDisplay(void)
{
    uint8_t minutes = seconds / 60;
    uint8_t sec = seconds % 60;

    display_data[0] = minutes / 10;
    display_data[1] = minutes % 10;
    display_data[2] = sec / 10;
    display_data[3] = sec % 10;

    display_data[4] = 18;
    display_data[5] = 18;
    display_data[6] = 18;
    display_data[7] = 18;
}

CY_ISR(Timer_Int_Handler2)
{
    Timer_1_ReadStatusRegister();

    FourDigit74HC595_sendOneDigit(led_counter,
display_data[led_counter], 0);

    led_counter++;

    if (led_counter > 7)
    {
        led_counter = 0;
    }

    ms_counter++;

    if (ms_counter >= 1000)
    {
        ms_counter = 0;

        if (seconds > 0)
        {
            seconds--;
            updateDisplay();
        }
    }
}

int main(void)
{
    CyGlobalIntEnable;

```

```

updateDisplay();

Timer_Int_StartEx(Timer_Int_Handler2);
Timer_1_Start();

for (;;)
{
}
}

```

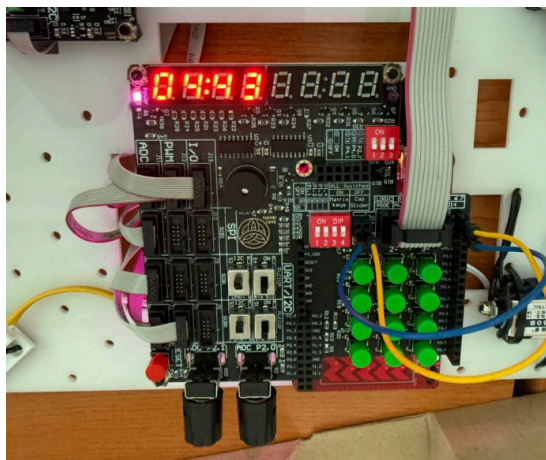
Результат: У результаті виконання програми на перших чотирьох семисегментних індикаторах реалізовано зворотний відлік часу від 05:00 до 00:00. Початкове значення часу задається змінною:

```
static uint16 seconds = 300;
```

Оскільки 300 секунд дорівнює 5 хвилинам, програма починає відлік саме з цього значення.

Функція *updateDisplay()* перетворює загальну кількість секунд у хвилини та секунди. Перші два індикатори показують хвилини, а третій і четвертий – секунди. Останні чотири індикатори очищуються значенням 18, яке відповідає порожньому символу.

У перериванні таймера виконується динамічне оновлення індикаторів. Змінна *ms_counter* використовується для підрахунку мілісекунд. Коли її значення досягає 1000, проходить одна секунда, після чого змінна *seconds* зменшується на одиницю.



Отже, завдання виконано: на перших чотирьох індикаторах реалізовано таймер зворотного відліку на 5 хвилин.

Висновки:

У ході виконання лабораторної роботи №5 було набуто навички роботи з динамічною індикацією, таймером, перериваннями та логічним аналізатором у середовищі *PSoC Creator*. Було реалізовано виведення чисел на семисегментний індикатор, декодування сигналів за допомогою логічного аналізатора, введення та перевірку паролю, циклічний зсув символів відповідно до варіанту, а також відлік 5 хвилин на перших чотирьох індикаторах. У результаті всі завдання лабораторної роботи були виконані, а робота з індикаторами, зсувними регістрами, таймером і перериваннями була успішно закріплена.